

Beyond Single Pass, Looping Through Time: KG-IRAG with Iterative Knowledge Retrieval

Ruiyi Yang

University of New South Wales
Sydney, NSW, Australia
ruiyi.yang@student.unsw.edu.au

Hao Xue

The Hong Kong University of Science and Technology
(Guangzhou)
Guangzhou, China
haoxue@hkust-gz.edu.cn

Imran Razzak

Mohamed Bin Zayed University of Artificial Intelligence
Abu Dhabi, UAE
imran.razzak@mbzuai.ac.ae

Flora D. Salim

University of New South Wales
Sydney, NSW, Australia
flora.salim@unsw.edu.au

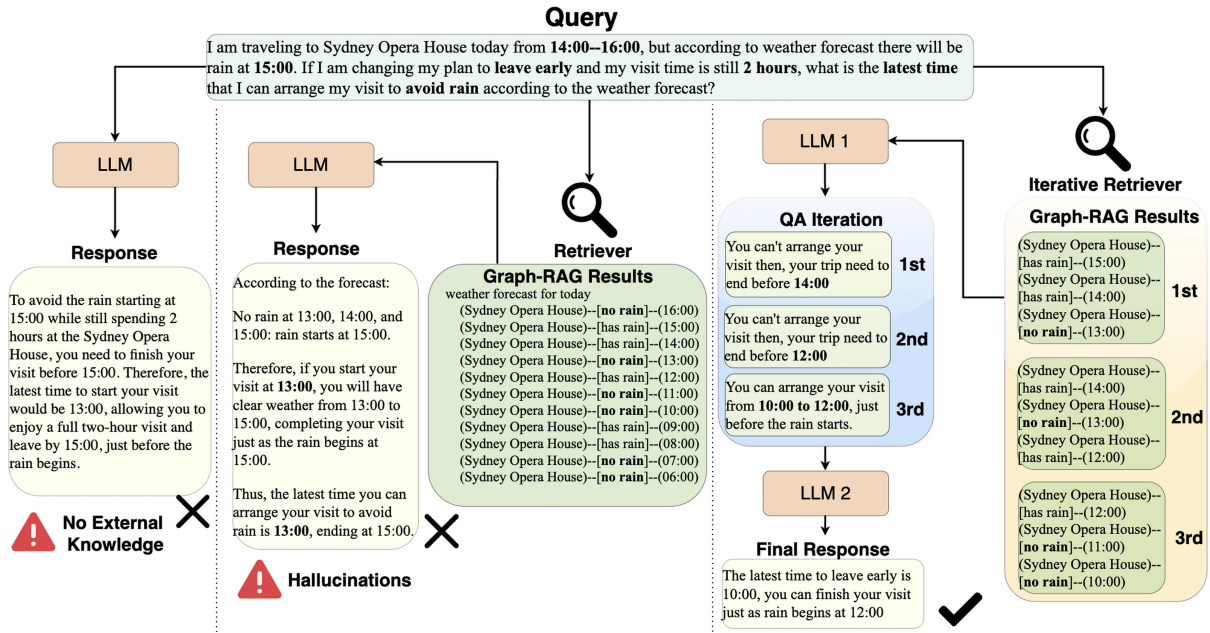


Figure 1: A comparison of LLM performance: Direct LLM output, RAG, and Iterative RAG. Without external knowledge, LLMs generate responses solely based on the provided context. However, when presented with excessive data retrieved by RAG (especially numerical information), LLMs are prone to generating hallucinated content.

Abstract

Retrieval-augmented generation (RAG) has improved large language models (LLMs) on knowledge-intensive tasks, yet most systems assume static facts and struggle when answers depend on serialized and dynamic data, like time—e.g., ordering events, aligning facts to valid intervals, or planning actions under evolving conditions. This paper presents **Knowledge-Graph Iterative Retrieval-Augmented Generation (KG-iRAG)**, a framework specialized for

temporal reasoning. KG-iRAG couples a time-aware planner with a knowledge graph (KG) to iteratively fetch and compose evidence along a temporal axis. Concretely, it (i) represents events and facts with explicit timestamps and validity intervals; (ii) propagates temporal constraints through iterative retrieval using operators; and (iii) verifies temporal consistency while refining intermediate hypotheses, enabling step-by-step deduction for queries that mix knowledge retrieval with inference. Across public temporal QA benchmarks, KG-iRAG consistently improves accuracy and calibration over strong RAG baselines while reducing unnecessary retrieval through targeted, constraint-guided steps. To stress-test real-time decision queries, three application-oriented datasets (*weatherQA-Irish*, *weatherQA-Sydney*, and *trafficQA-TFNSW*) are additionally constructed and tested alongside existing temporal benchmarks.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates.*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792745>

The results demonstrate that injecting temporal structure into KG-driven RAG yields robust gains on multi-step, time-dependent queries, advancing the state of temporal reasoning with LLMs.

CCS Concepts

• **Computing methodologies** → **Natural language generation; Information extraction.**

Keywords

Retrieval Augmented Generation, Knowledge Graph, Multi-agent, Temporal Reasoning

ACM Reference Format:

Ruiyi Yang, Hao Xue, Imran Razzak, and Flora D. Salim. 2026. Beyond Single Pass, Looping Through Time: KG-IRAG with Iterative Knowledge Retrieval. In *Proceedings of the ACM Web Conference 2026 (WWW '26), April 13–17, 2026, Dubai, United Arab Emirates*. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792745>.

1 Introduction

While State-of-the-art LLMs still suffered from limitations like fabricating fact or giving outdated answers for questions without answer or require latest data [22, 41], recent advances in Retrieval-Augmented Generation (RAG) offered a solution by integrating external knowledge sources like KGs with large language models (LLMs), the reliable and flexible supplementary knowledge helps LLMs generate more accurate replies to those queries that require domain-specific information [30, 40], without extra time for fine-tuning. As a variant of RAG, GraphRAG utilizes KGs to enhance the retrieval process through relational data. KGs capture intricate relationships between entities, allowing for more contextually rich predictions, thus offering more precise and contextually relevant responses from LLMs [9, 26, 34]. Although RAG achieves effectiveness in improving retrieval, they often fail to address the challenges posed in time-related tasks, such as planning trips or making decisions based on weather or traffic conditions. Dealing those queries, current RAG framework still faces several challenges:

C₁ : Accurate Retrieval LLMs need to generate a clear data retrieving plan, avoiding both insufficient(wrong answer) and redundant(waste of token) data recall [6].

C₂ : Hallucination Even with correct extra data input, facing too many data in prompts, especially numbers, LLMs tend to generate hallucinations and not following correct facts [13].

C₃ : Context-aware Reasoning LLMs need complex reasoning to analyze the data input, finding the accurate patterns behind numbers to get the final correct answers solving the temporal queries [17].

Figure 1 shows a scenario in which LLMs tend to generate hallucinations facing trip planning. To do such complex reasoning on huge amount of data input, existing RAG methods primarily focus on one-time knowledge retrieval rather than iterative retrieval. To address these limitations, this paper introduces a novel framework, **Knowledge Graph-Based Iterative Retrieval-Augmented Generation (KG-IRAG)**. KG-IRAG integrates KGs with an iterative reasoning process, enabling LLMs to solve problems by incrementally retrieving relevant data through multi-step queries and achieve

step-by-step reasoning. The proposed KG-IRAG framework is validated through experiments on datasets containing queries involving temporal dependencies, demonstrating its ability to significantly enhance LLM performance. Specifically, our key contributions are:

- **Iterative RAG Framework:** The new framework enables LLM reasoning in time-sensitive KG subgraphs through cyclic query refinement, offering a robust and efficient solution for addressing complex spatial temporal queries. The context-aware information retrieval via graph walks mirrors human-like reasoning patterns.
- **New datasets targeting complex temporal queries:** *weatherQA-Irish*, *weatherQA-Sydney* and *trafficQA-TFNSW* are designed to test LLM's ability to answer queries that require both **retrieving uncertain length of temporal information and doing mathematical reasoning**.

2 Related Work

2.1 Combination of Graphs with LLMs

The combinations of LLMs and graphs achieves mutual enhancement. The combination of LLMs with Graph Neural Networks (GNNs) has been shown to significantly improve the modeling capabilities of graph-structured data. LLMs have been shown to contribute to knowledge graph completion, aiding in downstream tasks such as node classification [5] and link prediction [29]. Additionally, LLMs play a pivotal role in knowledge graph creation, transforming source texts into graphs [9, 42]. LLMs also enhance performance in Knowledge Graph-based Question Answering (KBQA) tasks, which leverage external knowledge bases to answer user queries [7, 12, 35]. KBQA tasks includes text understanding and fact-checking [3, 32], and approaches are generally categorized into two types: Information Retrieval (IR)-based and Semantic Parsing (SP)-based. IR-based methods directly retrieve information from KG databases and use the returned knowledge to generate answers [20, 21], whereas SP-based methods generate logical forms for queries, which are then used for knowledge retrieval [2, 11]. Advanced techniques such as Chain of Knowledge (CoK) [24], G-Retriever [15], and Chain of Explorations [28] have been developed to enhance the precision and efficiency of data retrieval from KGs.

2.2 Retrieval-Augmented Generation (RAG)

RAG enhances the capabilities of LLMs by integrating external knowledge sources during the response generation process. Unlike traditional LLMs, which rely solely on their pre-trained knowledge, RAG enables models to access real-time or domain-specific information from external databases and knowledge sources. Recent studies have explored RAG from various perspectives, including the modalities of databases [43], model architectures, training strategies [10], and the diverse applications of RAG [14]. Effective evaluation of RAG systems requires attention to both the accuracy of knowledge retrieval and the quality of the generated responses [38].

Compared to traditional RAG systems, Graph Retrieval-Augmented Generation (GraphRAG) offers a distinct advantage by retrieving knowledge from graph databases, utilizing triplets as the primary data source [27]. Graph-structured data capture relationships between entities and offer structural information, enabling LLMs to interpret external knowledge more effectively [1, 16, 28].

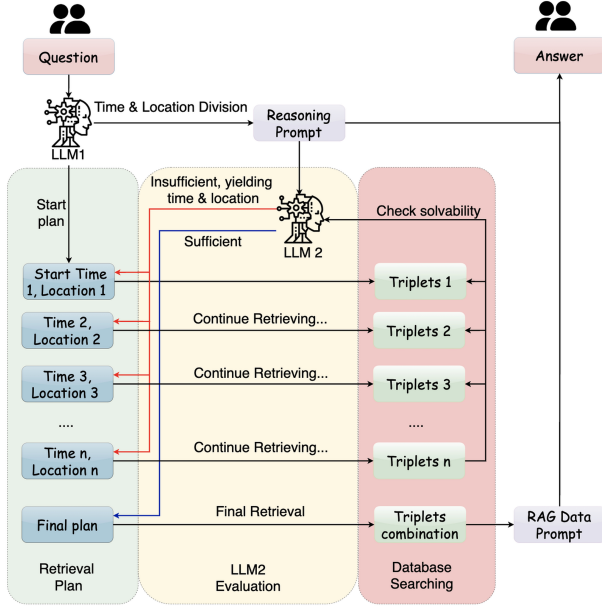


Figure 2: The KG-IRAG framework: LLM1 generates an initial retrieval plan and a reasoning prompt, guiding LLM2 through iterative retrievals or stopping to generate the final answer.

2.3 LLMs Temporal Reasoning

Recent advancements have increasingly focused on improving the temporal reasoning capabilities of LLMs. Temporal reasoning in natural language processing (NLP) typically falls into three categories: temporal expression detection and normalization [25], temporal relation extraction, and event forecasting [39]. The integration of temporal graphs has enabled LLMs to perform more effectively in tasks such as time comparison [37]. Several temporal QA datasets have been developed to test LLMs' temporal reasoning abilities, including TEMPLAMA [8, 33], TemporalWiki [18], and time-sensitive QA datasets [4]. By combining LLMs with temporal KGs [23, 36, 39], more accurate event forecasting has become possible.

While these methodologies have shown promising results, certain limitations remain: (1) few GraphRAG methods address queries highly dependent on temporal reasoning, and (2) no existing temporal QA dataset requires consecutive retrieval of uncertain amounts of data from a temporal knowledge base. The proposed KG-IRAG system aims to bridge these gaps by introducing a novel **iterative GraphRAG method** and developing datasets specifically focused on **dynamic temporal information retrieval**.

3 Beyond Single Pass, Looping Through Time: KG-IRAG Framework

This section presents the KG-IRAG framework for temporal reasoning. The key idea is to *anchor* retrieval on an explicit time point (or interval) and location, iteratively expand evidence along the temporal axis, and stop when a temporal-consistent answer can be verified. Two lightweight LLM agents collaborate throughout:

LLM1 proposes or updates the temporal/spatial anchors and the retrieval pattern; **LLM2** judges sufficiency and temporal consistency, and either requests another step or finalizes the answer.

3.1 Setup and Iterative Overview

Let q be a natural-language query whose answer depends on time. A reasoning state at iteration t is

$$S^{(t)} = (\tau^{(t)}, \ell^{(t)}, R^{(t)}, p^{(t)}) \quad (1)$$

where $\tau^{(t)}$ is the current time anchor (a timestamp or interval), $\ell^{(t)}$ is the location context, $R^{(t)}$ is the set of retrieved KG triples at step t , and $p^{(t)}$ is a concise retrieval pattern (slot-filled from the query, e.g., entity types, relation hints, required temporal relation such as BEFORE/DURING/AFTER).

Given a temporal KG D (Sec. 3.2), a time-aware retrieval operator Ψ materializes evidence around the anchors:

$$\begin{aligned} R^{(t)} &= \Psi(D; \tau^{(t)}, \ell^{(t)}, p^{(t)}) \\ &= \{ h \in D \mid \underbrace{\text{time}(h) \in \mathcal{W}(\tau^{(t)})}_{\text{temporal window}} \wedge \underbrace{\text{dist}(\text{loc}(h), \ell^{(t)}) \leq r}_{\text{spatial radius}} \\ &\quad \wedge \underbrace{\text{match}(h, p^{(t)})}_{\text{pattern}} \}. \end{aligned} \quad (2)$$

where $\mathcal{W}(\tau^{(t)})$ is a fixed or adaptive window around the anchor, r is a spatial radius, and match enforces simple structural filters (e.g., relation type, entity role) and the required temporal relation implied by $p^{(t)}$. LLM2 evaluates whether the accumulated evidence $R^{(\leq t)} = \bigcup_{i=0}^t R^{(i)}$ suffices to answer q and is temporally consistent. A simple, interpretable stop rule combining confidence and consistency is used:

$$\text{stop}^{(t)} = \underbrace{\left[\text{conf}(\text{LLM2}(q, R^{(\leq t)})) \geq \theta \right]}_{\text{sufficiency}} \wedge \underbrace{\left[\text{AllenConsistent}(R^{(\leq t)}) \right]}_{\text{interval constraints}}, \quad (3)$$

where $\text{AllenConsistent}(\cdot)$ is a lightweight check that the time intervals implied by $R^{(\leq t)}$ satisfy the expected BEFORE/AFTER/DURING/OVERLAP relations for q . If $\text{stop}^{(t)} = \text{false}$, LLM1 proposes the next anchors by scoring nearby time/location candidates using a utility that prefers *new* temporal coverage with low redundancy:

$$\begin{aligned} (\tau^{(t+1)}, \ell^{(t+1)}, p^{(t+1)}) &= \arg \max_{(\tau', \ell') \in \mathcal{N}_\tau \times \mathcal{N}_\ell} Q(\tau', \ell'), \\ Q(\tau', \ell') &= \underbrace{\text{gain}(q, R^{(\leq t)}, \tau', \ell')}_{\text{LLM1-estimated utility}} - \lambda \text{overlap}(R^{(\leq t)}, \tau', \ell'). \end{aligned} \quad (4)$$

where $\mathcal{N}_\tau/\mathcal{N}_\ell$ are local temporal and spatial neighborhoods (e.g., adjacent time windows / nearby stations). This keeps the math minimal while making the policy explicit.

Algorithm 1 KG-iRAG Time-Anchored Iterative Retrieval

Require: Query q , temporal KG D , thresholds $(\theta, T_{\max})$
Ensure: Final answer ans

```

1:  $(\tau^{(0)}, \ell^{(0)}, p^{(0)}) \leftarrow \text{LLM1\_Init}(q)$   $\triangleright$  initial anchors & pattern
2: for  $t = 0$  to  $T_{\max}$  do
3:    $R^{(t)} \leftarrow \Psi(D; \tau^{(t)}, \ell^{(t)}, p^{(t)})$   $\triangleright$  Eq. 2
4:    $(ok, \widehat{ans}) \leftarrow \text{LLM2\_Judge}(q, R^{(\leq t)})$ 
5:   if  $ok$  and  $\text{AllenConsistent}(R^{(\leq t)})$  then  $\triangleright$  Eq. 3
6:     return  $\widehat{ans}$ 
7:   end if
8:    $(\tau^{(t+1)}, \ell^{(t+1)}, p^{(t+1)}) \leftarrow \text{LLM1\_Update}(q, R^{(\leq t)})$   $\triangleright$  Eq. 4
9: end for
10: return  $\text{LLM2\_Fallback}(q, R^{(\leq T_{\max})})$ 

```

Practical notes. (i) $p^{(t)}$ is a short, human-readable template (entities/relations and required temporal predicate), kept stable unless LLM2 flags mismatch; (ii) $\mathcal{W}(\cdot)$ can shrink when confidence rises to avoid over-retrieval; (iii) $T_{\max}$ bounds latency; in practice 3–6 steps suffice for our tasks.

3.2 Temporal KG Construction

Data is presented as a directed, typed multigraph $D = (V, E)$ with explicit time:

- **Nodes** $V = V_{\text{time}} \cup V_{\text{loc}} \cup V_{\text{event}} \cup V_{\text{value}}$, where V_{time} holds time *points* or *intervals* $[t_s, t_e]$, V_{loc} are geo-entities (e.g., station/road), V_{event} are occurrences (e.g., rain, traffic), and V_{value} stores measured attributes (e.g. rain, vehicle count).
- **Edges** $E \subseteq V \times \mathcal{R} \times V$ with small, interpretable relation set $\mathcal{R}$ containing occursAt, atLocation, hasValue, before, after, during, overlaps, near.
Temporal relations follow Allen’s interval algebra; near is a symmetric spatial proximity.

Raw records d_i are normalized by a deterministic mapper Γ :

$$\Gamma(d_i) = \{(e_i, \text{occursAt}, \tau_i), (e_i, \text{atLocation}, \ell_i), (e_i, \text{hasValue}, v_i)\}$$

Storing validity intervals on V_{time} keeps retrieval simple (window checks reduce to interval tests), while derived before/during/overlaps edges allow fast constraint filtering inside Ψ (Eq. 2). This design makes time first-class, enables local temporal expansion around anchors, and supports lightweight consistency checks (Eq. 3) without heavy probabilistic machinery. Since application-oriented datasets (weather/traffic) are ingested by the same Γ ; no dataset-specific logic is required in the reasoning loop.

3.3 Triplet Retrieval

Given the current state $S^{(t)} = (\tau^{(t)}, \ell^{(t)}, R^{(t)}, p^{(t)})$, triplet retrieval instantiates the time-aware operator in Eq. 2. Intuitively, evidence is collected from a temporal window around $\tau^{(t)}$ and a spatial radius around $\ell^{(t)}$, while enforcing the structural/temporal pattern $p^{(t)}$.

Lightweight implementation. The retrieval window $\mathcal{W}(\tau^{(t)})$ and radius r start small and grow only when needed. To bound latency, the neighborhood depth is capped by $k_0 \in \{1, 2\}$ (local hops), with exact-dedup over previously seen triples to avoid re-fetch. A simple

priority order is used when multiple candidates satisfy the pattern: (i) exact time match $\Rightarrow$ (ii) within-window interval match $\Rightarrow$ (iii) nearest neighbor in time; ties are broken by spatial constraints on $\ell^{(t)}$. This yields compact batches $R^{(t)}$ that favor temporally on-point evidence. For temporal KGs, each $h \in R^{(t)}$ typically contains (event, occursAt, time) / (event, atLocation, loc) / (event, hasValue, v) and optionally before /after /during /overlaps edges derived from validity intervals. These derived edges permit fast pruning inside Ψ without additional model calls.

3.4 Iterative Reasoning

After accumulating $R^{(\leq t)} = \bigcup_{i=0}^t R^{(i)}$, LLM2 proposes a candidate answer and returns a sufficiency score together with a temporal-consistency verdict. Stopping follows Eq. 3: confidence must exceed θ and the Allen-style interval constraints implied by $R^{(\leq t)}$ must be satisfied.

If not sufficient, the controller updates anchors using Eq. 4. In practice, the utility prefers (a) new time coverage adjacent to $\tau^{(t)}$, (b) nearby locations along the same line/road, and (c) patterns that fill the missing predicate needed by the query (e.g., switch from DURING to BEFORE if a latest-feasible-time is sought). This induces a measured outward expansion in time and space, rather than a wide one-shot sweep.

Algorithm 2 Temporal Loop Controller (LLM1 + LLM2)

Require: Query q , KG D , thresholds $\theta, T_{\max}$

```

1:  $(\tau^{(0)}, \ell^{(0)}, p^{(0)}) \leftarrow \text{LLM1\_Init}(q)$ 
2: for  $t = 0$  to  $T_{\max}$  do
3:    $R^{(t)} \leftarrow \Psi(D; \tau^{(t)}, \ell^{(t)}, p^{(t)})$ 
4:    $(ok, \widehat{ans}) \leftarrow \text{LLM2\_Judge}(q, R^{(\leq t)})$ 
5:   if  $ok$  and  $\text{AllenConsistent}(R^{(\leq t)})$  then
6:     return  $\widehat{ans}$ 
7:   end if
8:    $(\tau^{(t+1)}, \ell^{(t+1)}, p^{(t+1)}) \leftarrow \text{LLM1\_Update}(q, R^{(\leq t)})$ 
9: end for
10: return “no answer”  $\triangleright$  budget reached

```

Interpretation. LLM2 acts as a temporal verifier: it accepts only when the proposed answer is both well-supported and interval-consistent. LLM1 acts as a planner: it shifts anchors to reduce uncertainty with minimal new retrieval.

3.5 Answer Generation

When the stop rule fires, the final answer is produced from the consolidated evidence $E = \text{Dedup}(R^{(\leq t)})$ together with the query q . A simple, transparent fusion is used: facts close to the *decisive time* receive higher weight.

$$ans = \text{LLM2_Synthesize}\left(q, \sum_{h \in E} \alpha(h) \cdot h\right), \quad (5)$$

$$\alpha(h) \propto \exp(-\gamma \cdot \text{dist_time}(h, t^*)) \cdot \mathbf{1}\{\text{match}(h, p^{(\leq t)})\}.$$

where t^* is selected from E as the time point/interval that satisfies the query’s temporal predicate (e.g., latest feasible departure), and γ controls temporal decay (default 0.5–1.0). The synthesis call returns both the answer and a short natural-language rationale citing the highest-weight facts. If no t^* can be identified or the retrieval budget κ is exceeded, the system returns “no answer”.

Robustness to noisy evidence. Before Eq. 5, a contradiction filter removes pairs of facts that violate the accepted interval constraints for q (e.g., mutually exclusive values at the same time). This keeps the fused context compact and consistent.

3.6 Why Not Single-Pass Retrieval?

Single-pass RAG must choose a large temporal window and spatial neighborhood *a priori*. In temporal KGs, the effective branching factor δ (neighbors per hop) compounds with the *dependency depth* d (how many anchored steps are needed to satisfy the temporal predicate), leading to a cost that scales as

$$\text{Cost}_{\text{single}} \approx \Theta(\delta^d), \quad (6)$$

or it risks missing critical intervals if the window is narrowed. By contrast, KG-IRAG executes T controlled steps with small local radius k_0 (typically $k_0 \leq 2$), yielding

$$\text{Cost}_{\text{iter}} \approx T \cdot \Theta(\delta^{k_0}), \quad \text{with } k_0 \ll d.$$

This reduces extraneous retrieval while preserving accuracy: anchors are moved only when evidence indicates where additional temporal coverage is most useful. Empirically, $T \in [3, 6]$ suffices for the studied tasks, avoiding the accuracy–efficiency dilemma inherent to single-pass sweeps.

3.7 Case Study

To further explain the details of ‘iteration’ in KG-IRAG, a case study was conducted by simulating an initial travel plan to Sydney Opera House, including defining whether the initial plan is valid, as well as how to define an optimal time to adjust the trip. Considering the following Q1, Q2 and related data:

Q1: ‘I plan to visit the Sydney Opera House from 13:00 to 15:00 on December 5th. Considering the weather, can I avoid rain during the trip?’

Q2: ‘If I can’t avoid the rain, what is the earliest time I can postpone my trip to avoid rain?’

Temporal series (Dec 5, Sydney Opera House). 13:00 cloudy; **13:30 rain**; 14:00 cloudy; 14:30 cloudy; 15:00 cloudy; **15:30 rain**; **16:00 rain**; 16:30 cloudy; 17:00 cloudy; 17:30 cloudy; 18:00 cloudy; 18:30 cloudy.

Setup. Slot length $\Delta = 1\text{h}$. The retrieval operator Ψ (Eq. 2) uses a temporal window aligned to the current anchor and enforces the pattern p (NO-RAIN for a contiguous window). LLM2 applies the stop rule (Eq. 3); otherwise, anchors are updated by Eq. 4 to expand coverage with minimal redundancy.

Q1 – Fixed window feasibility (13:00–15:00). Initialize $\tau^{(0)} = [13:00, 15:00]$, $\ell^{(0)} = \text{Opera House}$, $p^{(0)} = \text{NO-RAIN-IN-WINDOW}$. Applying Ψ retrieves evidence $R^{(0)}$ that includes **13:30 rain**. LLM2 judges insufficiency for NO-RAIN and returns **Q1 = No** (fails temporal predicate); the process stops for Q1.

Q2 – Earliest postponement with a 2h dry window. Now p requires a contiguous 2h NO-RAIN interval with the earliest feasible start time $\geq 13:30$. Iteration proceeds as follows (updates choose the closest future window that increases temporal coverage while avoiding overlap):

Step t	Anchor window $\tau^{(t)}$	Key evidence from Ψ	Decision (LLM2 / Controller)
0	[14:00, 16:00]	15:30 rain, 16:00 rain	Violates NO-RAIN; update by Eq. 4
1	[16:00, 18:00]	16:00 rain	Still violates; shift start forward
2	[16:30, 18:30]	16:30–18:30 all cloudy	Satisfies pattern; stop (Eq. 3)

The earliest feasible postponement that yields a 2h rain-free interval is therefore **16:30–18:30**. The controller favors [16:30, 18:30] at Step 2 because it is the first window starting after the last observed rain time (16:00) that also meets the 2h constraint without overlap-induced redundancy.

Answer synthesis. Upon stopping, evidence $E = \text{Dedup}(R^{(\leq t)})$ is fused by the temporal weighting in Eq. 5, where the decisive time t^* is the window start (16:30) satisfying the query predicate. The generated rationale cites the rain hits at 13:30/15:30/16:00 and the subsequent cloudy stretch 16:30–18:30.

4 Dataset Properties

4.1 Data Properties

Three application-oriented corpora are used to stress temporal reasoning under different sampling cadences:

- **weatherQA-Irish** (Jan 2017–Dec 2019): hourly records from Met Éireann, covering 25 stations across 15 counties.
- **weatherQA-Sydney** (Jan 2022–Aug 2024): weather observations collected every 30 minutes in Sydney.
- **trafficQA-TFNSW** (2015–2016): hourly traffic volumes from permanent counters/classifiers in Sydney.

Table 1: Corpus scale after normalization.

Irish Weather			
period	entities	relations	records (raw)
2017–2019	227,760	876,000	219,000
Sydney Weather			
period	entities	relations	records (raw)
2022–2024	332,433	559,673	279,837
TFNSW Traffic			
period	entities	relations	records (raw)
2015–2016	132,042	683,002	683,002

Shown in Table 6, during dataset creation, time is treated as a first-class entity (points or short intervals) to simplify window checks and downstream temporal constraints. Table 1 reports the corpus sizes after normalization. “Records (raw)” counts the ingested rows prior to KG mapping and deduplication. Weather corpora contain extra attributes (e.g., humidity, sunlight) and occasional duplicates;

these are dropped, so relations can exceed raw rows. The TFNSW corpus is already compact and fully utilized.

4.2 Question–Answer Generation

The QA design progressively increases temporal difficulty: **Q1** is a base detection on a fixed window; **Q2/Q3** require locating the nearest feasible window before/after an anchor time. This section only defines labels and generation policy; the iterative reasoning mechanics are covered in the Method section.

Sampling anchors. For each KG D , anchor times $\mathcal{T}_{\text{anchor}}$ are sampled uniformly within the temporal coverage. A trip duration Δt (e.g., 1–2 h) and a maximum search horizon L (e.g., 9–12 h) are fixed per dataset.

Q1: base detection on a fixed window. For anchor t , define the event-existence indicator

$$I_{\text{exist}}(t) = \mathbb{I}(\exists e \in D \text{ within } [t, t + \Delta t]).$$

Two phrasings are supported in the datasets: (i) *Detection* label $a_{\text{det}}(t) = I_{\text{exist}}(t)$; (ii) *Avoidability* label $a_{\text{avoid}}(t) = \neg I_{\text{exist}}(t)$. In practice, Q2/Q3 are instantiated only when the planned window is not avoidable, i.e., $I_{\text{exist}}(t) = 1$.

Q2/Q3: nearest feasible window before/after the anchor. Given the same duration Δt and horizon L ,

$$a_{\text{early}}(t) = \max \{t' < t \mid \nexists e \in [t', t' + \Delta t], t' > t - L\},$$

$$a_{\text{late}}(t) = \min \{t' > t \mid \nexists e \in [t', t' + \Delta t], t' < t + L\}.$$

If no such t' exists within the horizon, the label is set to NONE.

Algorithm 3 QA Pair Generation (per KG)

Require: KG D , #anchors M , duration Δt , horizon L

Ensure: QA set Q

```

1:  $Q \leftarrow \emptyset$ ,  $\mathcal{T}_{\text{anchor}} \leftarrow \text{UniformSample}(D.\text{times}, M)$ 
2: for each  $t \in \mathcal{T}_{\text{anchor}}$  do
3:   add  $(q_1(t), a_{\text{det/avoid}}(t))$ 
4:   if  $I_{\text{exist}}(t) = 1$  then
5:     compute  $a_{\text{early}}(t)$ ,  $a_{\text{late}}(t)$  within  $L$ 
6:     add  $(q_2(t), a_{\text{early}}(t))$ ,  $(q_3(t), a_{\text{late}}(t))$ 
7:   end if
8: end for
```

Numbers. Table 2 summarizes the distribution of Q1 labels and the availability of Q2/Q3 answers. For Q2/Q3, counts are reported only over anchors with $I_{\text{exist}}(t) = 1$ (i.e., the planned window initially violates the constraint).

Example (weatherQA–Sydney, 30-min cadence). Anchor: Location path: [PARRAMATTA NORTH → HOLSWORTHY CONTROL RANGE]; start time: 11-Mar-2024 04:00; trip duration $\Delta t = 4\text{h}$.

- **Q1 (avoidability).** “I am a resident in PARRAMATTA NORTH, and I plan to have a trip on 11-Mar-2024 04:00, passing [‘PARRAMATTA NORTH’, ‘HOLSWORTHY CONTROL RANGE’] in 4 hours, considering the weather, can I avoid rain during this time?”

Gold: False (rain at 04:00).

Table 2: QA set statistics. Q1 counts are over all anchors; Q2/Q3 counts are over anchors with $I_{\text{exist}}(t) = 1$.

Q1: abnormal event over $[t, t + \Delta t]$			
Dataset	#items	True (event)	False (no event)
weatherQA-Irish	600	393	207
weatherQA-Sydney	600	211	389
trafficQA-TFNSW	400	253	147

Q2&Q3 (conditioned on Q1=True)			
Dataset	seed cases	has answer	no answer
weatherQA-Irish	207	204	3
weatherQA-Sydney	389	339	50
trafficQA-TFNSW	147	135	12

- **Q2 (latest-before, leave early).** “Based on the above situation, if I cannot avoid rain, I can leave early, what is the latest time for me to leave early. If I can avoid rain, just answer ‘no need’, if there is no such time, just answer ‘no answer’.”

Gold: no answer (no rain-free 4h window entirely before 04:00).

- **Q3 (earliest-after, leave late).** “Based on the above situation, if I cannot avoid rain, I can leave late until there is no rain, what is the earliest time for me to leave late? If I can avoid rain, just answer ‘no need’, if there is no such time, just answer ‘no answer’.”

Gold: 2024-03-11T04:30:00 (04:30–08:30 all “No rain”).

4.3 Dynamic ProblemDecomposition

Q2/Q3 decompose a global temporal requirement (“find a feasible Δt window closest to the anchor”) into a sequence of fixed-window checks identical in structure to Q1. This yields a uniform labeling rule across datasets while naturally exercising the time-anchored iteration in method: anchors advance just beyond violating intervals, and feasibility is certified by simple interval tests. The design avoids dataset-specific heuristics yet produces queries that require genuine temporal reasoning rather than single-shot lookup.

5 Experiments

5.1 Evaluation

Tasks and datasets. Evaluation covers the three application datasets in Sec. 4.2 and one external temporal QA benchmark to show KG-iRAG’s usability on other cases. As a KG-based temporal QA set built on Wikidata with explicit/implicit/ordinal temporal categories and standard splits, **TimeQuestions** [19] (16,181 Qs) is adopted for out-of-domain benchmarking.

Standard Data (minimal evidence). Each question is paired with a *minimal* evidence set SD that suffices to answer it:

$$SD = \arg \min_{D \subseteq D_{\text{all}}} \text{s.t. } D \text{ solves the query,} \quad (7)$$

$$\forall d \in SD, (SD \setminus \{d\}) \text{ does not solve the query.} \quad (8)$$

This ensures fair comparison across formats (raw table, verbalized text, KG triples) without rewarding extraneous context.

Metrics. Reporting uses **Accuracy (Acc)** and **macro-F1** for answer quality, and a Jaccard-style **Hit Rate (HR)** for retrieval usefulness (share of retrieved items that intersect *SD*). Hallucination is explicitly measured in Eq. 10.

Hallucination. Hallucination denotes content not supported by the provided evidence (or violating the accepted temporal constraints). The indicator and rate are

$$\text{hal}(\text{answer}_i, \text{truth}_i) = \begin{cases} 1 & \text{hallucination detected} \\ 0 & \text{otherwise} \end{cases} \quad (9)$$

$$\text{Hallucination}(\%) = \frac{\sum_{i=0}^N \text{hal}(\text{answer}_i, \text{truth}_i)}{N} = 1 \quad (10)$$

Operationally: for **Q1**, any incorrect yes/no constitutes hallucination; for **Q2/Q3**, hallucination is flagged if (a) the predicted time window is not in evidence, (b) the predicted abnormal event time is wrong, or (c) the controller fails to stop despite meeting temporal constraints. For **Q2/Q3** on each dataset, 50 items are randomly sampled for manual verification.

Models and baselines. LLMs: Llama-3-8B-Instruct, GPT-3.5-turbo-0125, GPT-4o-mini-2024-07-18, GPT-4o-2024-08-06, Falcon-H1-7b [44]. Baselines: (i) **Single-pass Graph-RAG** (one-shot selection of a fixed temporal window, no iteration); (ii) **KG-RAG** [28] with three exploration steps following the Chain-of-Explorations retrieval plan; (iii) **Think-on-Graph** [31] with knowledge traceability and knowledge correctness by leveraging LLMs reasoning and expert feedback. (iv) **KG-iRAG** (proposed).

For **TimeQuestions**, official train/dev/test splits and Wikidata snapshot are used as provided; KG-iRAG and baselines interface with the KG via the same retrieval API and report EM/F1 only (no *SD*/HR, as gold minimal evidence is not supplied).

5.2 Experiment Settings

Stage A — Format sensitivity without iteration. To isolate LLM temporal reasoning *given correct evidence*, each question is paired with its *SD* and rendered in three formats: $\{d_{\text{raw}}, d_{\text{ext}}, d_{\text{triplet}}\}$. These are injected into a fixed answer template and fed to each LLM without retrieval/iteration. This diagnoses whether representation alone affects temporal reasoning on our datasets.

Stage B — End-to-end retrieval and QA. Baselines and KG-iRAG are run end-to-end on all datasets. For KG-RAG (CoE), the exploration depth is capped at 3 steps to match the temporal span of typical questions; for ReAct-KG, the agent can interleave short reasoning traces with time-bounded KG actions but has no access to our specialized stop rule; for Single-pass, a fixed window is chosen once from the anchor. KG-iRAG uses the same underlying LLMs for *LLM1* (planner) and *LLM2* (verifier) as in Fig. 2. EM/F1/HR and hallucination are reported per dataset.

6 Result

6.1 Result analysis

6.1.1 Direct inputs (no retrieval/iteration). Across models and datasets, structured **triplets** generally yield the best accuracy in Table 8. Especially for temporally grounded queries (*Q2/Q3*), triplets are consistently stronger: e.g., on *Irish Q2*, GPT-4o reaches 0.396 (triplets)

vs. 0.3865/0.3314; on *Sydney Q2*, GPT-4o improves to 0.347 (triplets); on *TFNSW Q3*, GPT-4o attains 0.7279 (triplets). These trends indicate that explicitly exposing *time–location–event* structure reduces spurious reasoning and helps align constraints.

6.1.2 RAG comparison on Q1. In Table 3, standard Graph-RAG, KG-RAG, ToG and KG-iRAG perform similarly on *Q1*. This is expected: when the window is fixed and the predicate is a binary existence test, one-pass retrieval already provides adequate evidence, and iterative control offers limited additional benefit.

6.1.3 RAG comparison on Q2/Q3 (temporal search). On temporally grounded questions, KG-iRAG consistently ranks ahead of all baselines in Table 3. Compared with other baselines, which contains premature stops or lacks explicit temporal windows, KG-iRAG avoids both over- and under-retrieval by updating anchors just beyond violating events and verifying sufficiency with interval consistency before answering. The same behavior is observed on the external *TimeQuestions* benchmark: without dataset-specific heuristics, the time-anchored controller and lightweight consistency checks transfer directly to Wikidata timestamps/validity intervals, preserving the ordering among methods. This suggests the approach is not tailored to the in-house corpora and can be applied to other temporal KGQA datasets with minimal adaptation (i.e., a shared retrieval API and time-normalized triples).

6.1.4 Hallucination (definition-based auditing). Manual audits (50 items per dataset for *Q2/Q3*) following the definition in Eq. 9 show a clear ordering: *Single-pass Graph-RAG* has the highest hallucination, driven by oversized contexts when the model requests maximal horizons; *KG-RAG* reduces hallucination but is vulnerable to under-retrieval on longer spans; **KG-iRAG** yields low hallucination, attributable to (i) minimal-evidence retrieval around anchors and (ii) explicit interval-consistency checks before answer synthesis. For *Q1*, hallucination rates largely mirror EM errors, consistent with the binary definition.

6.2 Ablation Study

Two variants are derived from KG-iRAG to diagnose which components drive temporal search quality. *Limited-Recall* caps the number of iterations to a small fixed budget (2-iteration at most) and disables adaptive window growth, forcing the controller to answer after a short exploratory. *No-Sufficiency-Check* removes the stop rule in Eq. 3; the controller executes a fixed number of steps and synthesizes an answer without verifying temporal consistency. Experiments are run on *weatherQA-Irish* (hourly) and *weatherQA-Sydney* (30-min), focusing on *Q2/Q3*.

Both variants underperform the full system on earliest / earliest-after and latest / earliest-before queries. Capping iterations induces under-coverage near the decisive boundary, particularly when the first feasible window lies just beyond the last violating event; answers then lean conservative for “latest-before” and overly optimistic for “earliest-after”. Removing the sufficiency check produces the opposite failure mode: retrieval drifts into redundant windows, the fused context becomes harder to calibrate, and hallucination rises when the synthesis step commits to unsupported intervals. The effect is more visible in higher-frequency data where windows

Table 3: Performance comparison of different methods on KGQA benchmarks with KG(Triplets) input. The best and second-best methods are denoted.

Method	Overall Results													
	WeatherQA-Irish				WeatherQA-Sydney				TrafficQA-TFNSW				TimeQuestion	
	Q2-Acc	Q2-F1	Q3-Acc	Q3-F1	Q2-Acc	Q2-F1	Q3-Acc	Q3-F1	Q2-Acc	Q2-F1	Q3-Acc	Q3-F1	Acc	F1
LLM methods (no KG retrieval)														
Llama3-8b	0.009	N/A	0.048	N/A	0.141	N/A	0.149	N/A	0.061	N/A	0.087	N/A	0.178	N/A
GPT-3.5-turbo	0.034	N/A	0.034	N/A	0.129	N/A	0.136	N/A	0.054	N/A	0.095	N/A	0.258	N/A
GPT-4o-mini	0.014	N/A	0.058	N/A	0.123	N/A	0.149	N/A	0.087	N/A	0.102	N/A	0.327	N/A
GPT-4o	0.043	N/A	0.053	N/A	0.126	N/A	0.141	N/A	0.068	N/A	0.109	N/A	0.354	N/A
Falcon-H1-7B-Instruct	0.048	N/A	0.072	N/A	0.129	N/A	0.147	N/A	0.075	N/A	0.116	N/A	0.348	N/A
KG+LLM methods														
Graph-RAG	0.208	0.673	0.357	0.684	0.226	0.514	0.25	0.503	0.361	0.694	0.565	0.709	0.693	0.734
KG-RAG [28]	0.435	<u>0.857</u>	0.575	0.839	0.378	0.785	0.362	0.793	<u>0.537</u>	<u>0.878</u>	0.776	0.869	0.717	0.783
ToG [31]	<u>0.449</u>	0.842	<u>0.584</u>	<u>0.857</u>	<u>0.411</u>	<u>0.833</u>	<u>0.398</u>	<u>0.824</u>	0.531	0.861	<u>0.789</u>	<u>0.893</u>	0.742	0.812
KG-iRAG	0.459	0.875	0.604	0.88	0.429	0.852	0.411	0.869	0.558	0.886	0.803	0.914	0.781	0.824

Table 4: Ablation on GPT-4o for temporal search (Q2/Q3). Acc/F1; arrows indicate change vs Full.

Variant	Task	weatherQA-Irish			Task	weatherQA-Sydney		
		Acc	F1			Acc	F1	
Full KG-iRAG	Q2	0.459	0.875		Q2	0.429	0.852	
Limited-Iteration	Q2	0.353	0.691		Q2	0.326	0.614	
No-Sufficiency-Check	Q2	0.391	0.733		Q2	0.362	0.702	
Full KG-iRAG	Q3	0.604	0.880		Q3	0.411	0.869	
Limited-Recall	Q3	0.488	0.626		Q3	0.296	0.598	
No-Sufficiency-Check	Q3	0.541	0.785		Q3	0.347	0.703	

are narrower and violations recur in short succession, as in the Sydney corpus.

6.3 Latency Analysis on TimeQuestions

End-to-end wall-clock latency per question is measured on *TimeQuestions*, comparing KG-iRAG with ToG under the same LLM backbone, KG backend, and token budgets. Latency includes planning, retrieval calls, and answer generation, with caching disabled.

Table 5: Latency on *TimeQuestions* with GPT-4o (same KG backend, token budgets, caching disabled). All values are estimated under a unified setup.

Method	Median (s)	Avg # LLM Calls	Avg # KG Calls
KG-iRAG(Ours)	4.9	4.2	2.1
KG-RAG	5.6	4.6	3.3
ToG	12.8	6.5	8.4

KG-iRAG exhibits lower latencies compared with other baselines. Anchors converge in a small number of steps, and the stop rule halts early once the required interval constraints are satisfied, keeping retrieval batches compact. ToG’s iterative planning remains competitive on multi-hop relational paths, but when queries hinge

on locating the nearest feasible time window, beam expansions and re-planning loops introduce additional tool calls and longer traces, which accumulates latency.

The observed latency pattern aligns with results on the in-house temporal datasets: when answers depend on verifying a short interval around an anchor rather than exploring long relational paths, time-anchored iteration with verifiable stopping maintains accuracy while avoiding prolonged interaction cycles. Consequently, the controller design transfers to other temporal KGQA settings with minimal adaptation, delivering both quality and efficiency without dataset-specific heuristics.

7 Conclusions

In this paper, a new RAG framework is proposed, i.e., Knowledge Graph-Based Iterative Retrieval-Augmented Generation(**KG-IRAG**), which enhances the integration of KG with LLMs through iterative reasoning and retrieval. Unlike traditional RAG methods, KG-IRAG flexibly and effectively employs a step-by-step retrieval mechanism that guides LLMs in determining when to stop exploration, significantly improving response accuracy. To evaluate the framework’s effectiveness, three new datasets, **weatherQA-Irish**, **weatherQA-Sydney**, and **trafficQA-TFNSW**, have been introduced. These datasets involve real-world scenarios, designed to test the ability of LLMs to handle time-sensitive and event-based queries requiring both temporal reasoning and logical inference. Experimental results with proposed benchmark and general temporal ones demonstrate that KG-IRAG excels in complex reasoning tasks by generating more accurate and efficient responses.

Acknowledgments

This research was partially supported by the Technology Innovation Institute (TII), Abu Dhabi, United Arab Emirates, which conducts cutting-edge applied research in artificial intelligence and advanced technologies. This research also includes computations using the computational cluster Wolfpack supported by School of Computer Science and Engineering at UNSW Sydney. Finally, This research used the NVIDIA H100 cluster provided by SHARON AI.

References

- [1] Diego Bustamante and Hideaki Takeda. 2024. SPARQL Generation with Entity Pre-trained GPT for KG Question Answering. *arXiv preprint arXiv:2402.00969* (2024).
- [2] Abir Chakraborty. 2024. Multi-hop Question Answering over Knowledge Graphs using Large Language Models. *arXiv preprint arXiv:2404.19234* (2024).
- [3] Wenhu Chen, Hongmin Wang, Jianshu Chen, Yunkai Zhang, Hong Wang, Shiyang Li, Xiyu Zhou, and William Yang Wang. 2019. Tabfact: A large-scale dataset for table-based fact verification. *arXiv preprint arXiv:1909.02164* (2019).
- [4] Wenhu Chen, Xinyi Wang, and William Yang Wang. 2021. A dataset for answering time-sensitive questions. *arXiv preprint arXiv:2108.06314* (2021).
- [5] Zhikai Chen, Haitao Mao, Hongzhi Wen, Haoyu Han, Wei Jin, Haiyang Zhang, Hui Liu, and Jiliang Tang. 2023. Label-free node classification on graphs with large language models (llms). *arXiv preprint arXiv:2310.04668* (2023).
- [6] Florin Cucanasu, Giovanni Trappolini, Federico Siciliano, Simone Filice, Cesare Campagnano, Yoelle Maarek, Nicola Tonello, and Fabrizio Silvestri. 2024. The power of noise: Redefining retrieval for rag systems. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 719–729.
- [7] Wanyun Cui, Yanghua Xiao, Haixun Wang, Yangqiu Song, Seung-won Hwang, and Wei Wang. 2019. KBQA: learning question answering over QA corpora and knowledge bases. *arXiv preprint arXiv:1903.02419* (2019).
- [8] Bhuwan Dhingra, Jeremy R Cole, Julian Martin Eisenschlos, Daniel Gillick, Jacob Eisenstein, and William W Cohen. 2022. Time-aware language models as temporal knowledge bases. *Transactions of the Association for Computational Linguistics* 10 (2022), 257–273.
- [9] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024).
- [10] Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A survey on rag meeting llms: Towards retrieval-augmented large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 6491–6501.
- [11] Haishuo Fang, Xiaodan Zhu, and Iryna Gurevych. 2024. DARA: Decomposition-Alignment-Reasoning Autonomous Language Agent for Question Answering over Knowledge Graphs. *arXiv preprint arXiv:2406.07080* (2024).
- [12] Bin Fu, Yunqi Qiu, Chengguang Tang, Yang Li, Haiyang Yu, and Jian Sun. 2020. A survey on complex question answering over knowledge base: Recent advances and challenges. *arXiv preprint arXiv:2007.13069* (2020).
- [13] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. 2023. Enabling large language models to generate text with citations. *arXiv preprint arXiv:2305.14627* (2023).
- [14] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* (2023).
- [15] Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. *arXiv preprint arXiv:2402.07630* (2024).
- [16] Yuntong Hu, Zhihan Lei, Zheng Zhang, Bo Pan, Chen Ling, and Liang Zhao. 2024. GRAG: Graph Retrieval-Augmented Generation. *arXiv preprint arXiv:2405.16506* (2024).
- [17] Jie Huang and Kevin Chen-Chuan Chang. 2022. Towards reasoning in large language models: A survey. *arXiv preprint arXiv:2212.10403* (2022).
- [18] Joel Jang, Seonghyeon Ye, Changho Lee, Sohee Yang, Joongbo Shin, Janghoon Han, Gyeonghun Kim, and Minjoon Seo. 2022. Temporalwiki: A lifelong benchmark for training and evaluating ever-evolving language models. *arXiv preprint arXiv:2204.14211* (2022).
- [19] Zhen Jia, Soumajit Pramanik, Rishiraj Saha Roy, and Gerhard Weikum. 2021. Complex temporal question answering on knowledge graphs. In *Proceedings of the 30th ACM international conference on information & knowledge management*. 792–802.
- [20] Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, Yang Song, Chen Zhu, Hengshu Zhu, and Ji-Rong Wen. 2024. Kg-agent: An efficient autonomous agent framework for complex reasoning over knowledge graph. *arXiv preprint arXiv:2402.11163* (2024).
- [21] Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, and Ji-Rong Wen. 2022. Unikgqa: Unified retrieval and reasoning for solving multi-hop question answering over knowledge graph. *arXiv preprint arXiv:2212.00959* (2022).
- [22] Nikhil Kandpal, Haikang Deng, Adam Roberts, Eric Wallace, and Colin Raffel. 2023. Large language models struggle to learn long-tail knowledge. In *International Conference on Machine Learning*. PMLR, 15696–15707.
- [23] Dong-Ho Lee, Kian Ahrabian, Woojeong Jin, Fred Morstatter, and Jay Pujara. 2023. Temporal knowledge graph forecasting without knowledge using in-context learning. *arXiv preprint arXiv:2305.10613* (2023).
- [24] Xingxuan Li, Ruochen Zhao, Yew Ken Chia, Bosheng Ding, Shafiq Joty, Soujanya Poria, and Lidong Bing. 2023. Chain-of-knowledge: Grounding large language models via dynamic knowledge adapting over heterogeneous sources. *arXiv preprint arXiv:2305.13269* (2023).
- [25] Zechen Li, Shohreh Deldari, Linyao Chen, Hao Xue, and Flora D Salim. 2024. Sensorllm: Aligning large language models with motion sensors for human activity recognition. *arXiv preprint arXiv:2410.10624* (2024).
- [26] Nicholas Matsumoto, Jay Moran, Hyunjun Choi, Miguel E Hernandez, Mythreye Venkatesan, Paul Wang, and Jason H Moore. 2024. KRAGEN: a knowledge graph-enhanced RAG framework for biomedical problem solving using large language models. *Bioinformatics* 40, 6 (2024).
- [27] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2024. Graph retrieval-augmented generation: A survey. *arXiv preprint arXiv:2408.08921* (2024).
- [28] Diego Sanmartin. 2024. KG-RAG: Bridging the Gap Between Knowledge and Creativity. *arXiv preprint arXiv:2405.12035* (2024).
- [29] Dong Shu, Tianle Chen, Mingyu Jin, Yiting Zhang, Mengnan Du, and Yongfeng Zhang. 2024. Knowledge Graph Large Language Model (KG-LLM) for Link Prediction. *arXiv preprint arXiv:2403.07311* (2024).
- [30] Shamane Siriwardhana, Rivindu Weerasekera, Elliott Wen, Tharindu Kalu-arachchi, Rajib Rana, and Suranga Nanayakkara. 2023. Improving the domain adaptation of retrieval augmented generation (RAG) models for open domain question answering. *Transactions of the Association for Computational Linguistics* 11 (2023), 1–17.
- [31] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M Ni, Heung-Yeung Shum, and Jian Guo. 2023. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. *arXiv preprint arXiv:2307.07697* (2023).
- [32] Suryavardan Suresh, Anku Rani, Parth Patwa, Aishwarya Reganti, Vinija Jain, Aman Chadha, Amitava Das, Amit Sheth, and Asif Ekbal. 2024. Overview of Factify5WQA: Fact Verification through 5W Question-Answering. *arXiv preprint arXiv:2410.04236* (2024).
- [33] Qingyu Tan, Hwee Tou Ng, and Lidong Bing. 2023. Towards benchmarking and improving the temporal reasoning capability of large language models. *arXiv preprint arXiv:2306.08952* (2023).
- [34] Junde Wu, Jiayuan Zhu, and Yunli Qi. 2024. Medical graph rag: Towards safe medical large language model via graph retrieval-augmented generation. *arXiv preprint arXiv:2408.04187* (2024).
- [35] Peiyun Wu, Xiaowang Zhang, and Zhiyong Feng. 2019. A survey of question answering over knowledge base. In *Knowledge Graph and Semantic Computing: Knowledge Computing and Language Understanding: 4th China Conference, CCKS 2019, Hangzhou, China, August 24–27, 2019, Revised Selected Papers 4*. Springer, 86–97.
- [36] Yuwei Xia, Ding Wang, Qiang Liu, Liang Wang, Shu Wu, and Xiaoyu Zhang. 2024. Enhancing temporal knowledge graph forecasting with large language models via chain-of-history reasoning. *arXiv preprint arXiv:2402.14382* (2024).
- [37] Siheng Xiong, Ali Payani, Ramana Kompella, and Faramarz Fekri. 2024. Large language models can learn temporal reasoning. *arXiv preprint arXiv:2401.06853* (2024).
- [38] Hao Yu, Aoran Gan, Kai Zhang, Shiwei Tong, Qi Liu, and Zhaofeng Liu. 2024. Evaluation of Retrieval-Augmented Generation: A Survey. *arXiv preprint arXiv:2405.07437* (2024).
- [39] Chenhan Yuan, Qianqian Xie, Jimin Huang, and Sophia Ananiadou. 2024. Back to the future: Towards explainable temporal reasoning with large language models. In *Proceedings of the ACM on Web Conference 2024*. 1963–1974.
- [40] Tianjun Zhang, Shishir G Patil, Naman Jain, Sheng Shen, Matei Zaharia, Ion Stoica, and Joseph E Gonzalez. 2024. Raft: Adapting language model to domain specific rag. *arXiv preprint arXiv:2403.10131* (2024).
- [41] Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, et al. 2023. Siren’s song in the AI ocean: a survey on hallucination in large language models. *arXiv preprint arXiv:2309.01219* (2023).
- [42] Yuzhe Zhang, Yipeng Zhang, Yidong Gan, Lina Yao, and Chen Wang. 2024. Causal graph discovery with retrieval-augmented generation based large language models. *arXiv preprint arXiv:2402.15301* (2024).
- [43] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, and Bin Cui. 2024. Retrieval-augmented generation for ai-generated content: A survey. *arXiv preprint arXiv:2402.19473* (2024).
- [44] Jingwei Zuo, Maksim Velikanov, Ilyas Chahed, Younes Belkadda, Dhia Eddine Khayem, Guillaume Kunsch, Hakim Hacid, Hamza Yous, Brahim Farhat, Ibrahim Khadraoui, et al. 2025. Falcon-h1: A family of hybrid-head language models redefining efficiency and performance. *arXiv preprint arXiv:2507.22448* (2025).

A Notations

Important notations are listed in Table 7.

Table 6: Attributes used for KG construction and QA labeling.

Irish Weather	
attribute	explanation
date	date/time of record (hourly)
county	county name
station ID	station identifier
station	station name
rain	rainfall volume per hour
Sydney Weather	
attribute	explanation
date	date/time of record (30-min)
station ID	station identifier
station	station name
rain	rainfall volume per 30 minutes
TFNSW Traffic	
attribute	explanation
date	date/time of record (hourly)
direction	cardinal direction (northbound/southbound)
volume	traffic count per hour

B Theoretical Analysis of Iterative Retrieval

B.1 Coverage necessity for single-pass windows

Definition B.1 (Nearest feasible offset). Given an anchor window $[t_0, t_0 + \Delta t]$ and a feasibility predicate (e.g., *no event during the window*), let $d^* \geq 0$ be the smallest number of window shifts (each of size Δt or a fixed stride) required so that the shifted window $[t_0 + d^* \cdot \text{stride}, t_0 + d^* \cdot \text{stride} + \Delta t]$ satisfies the predicate.

LEMMA 1 (SINGLE-PASS COVERAGE CONDITION). *Any single-pass retriever that inspects a temporal radius w (in the same unit as d^*) can succeed only if $w \geq d^*$. If a dataset contains instances with $d^* > w$, the single-pass procedure necessarily misses the first feasible window unless it increases w .*

SKETCH. A single-pass procedure fixed at radius w returns evidence confined to windows whose start offsets lie in $[-w, +w]$ around the anchor. The earliest feasible window starts at offset d^* . If $d^* > w$, that window is outside the inspected range and thus cannot be certified without enlarging the radius. $\square$

B.2 Cost scaling: single-pass vs. iterative

Let $\text{Cost}_{\text{temp}}(w)$ denote the temporal scan cost for radius w (linear in w), and let graph-expansion cost at hop k scale as $\Theta(\delta^k)$ with effective branching $\delta > 1$ when joining metadata, nearby entities, or derived edges. A single-pass strategy that guarantees success on all instances up to offset d^* must set $w \geq d^*$, yielding

$$\text{Cost}_{\text{single}} = \Theta(\text{Cost}_{\text{temp}}(d^*)) + \Theta(\delta^k).$$

In contrast, the time-anchored iterative policy uses small fixed $k_0 \ll k$ and advances anchors in $T \approx d^*$ bounded steps until the feasibility predicate holds:

$$\text{Cost}_{\text{iter}} = T \cdot (\Theta(\text{Cost}_{\text{temp}}(1)) + \Theta(\delta^{k_0})).$$

Since $\text{Cost}_{\text{temp}}$ is linear and δ^k grows super-linearly in k for $\delta > 1$, using a small per-step hop k_0 avoids the exponential blow-up required by a single-pass guarantee at large d^* while still covering the necessary offset through controlled iteration.

B.3 Precision monotonicity with window size

PROPOSITION 1 (CONTEXT DILUTION). *Assume irrelevant facts arrive at an average rate λ_{irr} per temporal unit near the anchor, while the number of relevant facts required to certify feasibility is bounded by a constant c (e.g., a small set of interval facts). The expected precision of a single-pass window of radius w then satisfies*

$$\mathbb{E}[\text{Precision}(w)] = \frac{c}{c + \Theta(\lambda_{\text{irr}} \cdot w)},$$

which is non-increasing in w . Iterative retrieval that stops immediately upon feasibility keeps w small at the stopping step and therefore attains precision at least as high as any single-pass run that uses a larger w to guarantee coverage.

SKETCH. Under the assumption, relevant mass is c while irrelevant mass grows proportionally with window size. Hence precision decreases with w . The iterative stop rule halts at the smallest window that satisfies feasibility (via AllenConsistent), bounding the final context and preventing further dilution. $\square$

Consequence. The pair of properties—(i) coverage through measured anchor advances and (ii) bounded final window via a verifiable stop rule—explains the empirical gains of KG-iRAG on Q2/Q3: it achieves the necessary temporal reach without incurring the single-pass cost at large radii and avoids context dilution that elevates hallucination.

C Experiment on QA datasets-Direct Data Input

In the first round of experiments, four LLMs are used to test how LLMs can directly answer the constructed three QA datasets (*weatherQA-Irish*, *weatherQA-Sydney*, and *trafficQA-TFNSW*), with some settings:

- 1) To ensure LLMs are not interrupted by useless information, input prompts only contain questions as well as **least needed data**.
- 2) To test the efficiency of data input, three formats of data are used: raw data (table) format, text data (by transferring data into text description), and triplet format (KG structure). Tests using three formats of data are done separately.
- 3) The final answer are compared directly with correct answers, the Accuracy(Acc) value is shown in table 8.

D Experiment on QA datasets-Different RAG Methods

In second stage of experiments, KG-IRAG is compared with Graph-RAG and KG-RAG [28]. None data are provided in the beginning, data retrieval plan are generated based on different framework. The Accuracy, like last stage of experiment, is the result of comparison between generated final answer and true answer. F1 Score and Hit Rate focus more on the whether the retrieval is both enough and non-excessive. Hallucinations are judged based on answers generated by LLMs under different framework. For Q1, F1 Score and

Table 7: Key notations used in KG-iRAG.

Symbol	Meaning
$S^{(t)} = (\tau^{(t)}, \ell^{(t)}, R^{(t)}, p^{(t)})$	Reasoning state at iteration t (time anchor, location, retrieved triples, pattern)
$\Psi(D; \tau, \ell, p)$	Time-aware retrieval operator around (τ, ℓ) under pattern p (Eq. 2)
$\mathcal{W}(\tau, r)$	Temporal window around anchor τ ; spatial radius
AllenConsistent($\cdot$)	Interval-consistency check used in the stop rule (Eq. 3)
LLM1 / LLM2	Planner (anchor update) / Verifier (sufficiency + synthesis)
k_0	Local neighborhood hop cap per retrieval (typically 1–2)
$T_{\max}$	Maximum iterations (latency budget)
SD	Standard/minimal evidence for a question (Sec. 4.2)
$\Delta t, L$	Trip duration and search horizon for Q2/Q3 labels
δ	Effective branching factor for graph expansion beyond the time line
d^*	Temporal offset (in window-steps) from initial anchor to the nearest feasible window

Table 8: Comparison of Direct Data Inputs on Three Datasets Based on Accuracy(Acc)

Data		Llama-3-8b			GPT-3.5-turbo			GPT-4o-mini			GPT-4o		
		Raw	Text	Triplet	Raw	Text	Triplet	Raw	Text	Triplet	Raw	Text	Triplet
weatherQA-Irish	Q1	0.905	0.9016	0.9316	0.9483	0.9417	0.9783	0.9817	0.9833	0.9933	0.9950	0.9967	1
	Q2	0.1159	0.1159	0.1256	0.14	0.0966	0.1111	0.1932	0.1498	0.2415	0.3865	0.3314	0.396
	Q3	0.3233	0.343	0.3382	0.2946	0.2995	0.314	0.4203	0.4348	0.4492	0.4831	0.5072	0.5169
weatherQA-Sydney	Q1	0.8867	0.855	0.8683	0.9117	0.7083	0.9383	1	0.9667	0.9717	0.9867	0.9667	0.99
	Q2	0.1542	0.1285	0.1491	0.1259	0.162	0.1722	0.2314	0.2596	0.3033	0.2545	0.2879	0.347
	Q3	0.1722	0.1517	0.1877	0.1671	0.2108	0.2005	0.2416	0.2699	0.2699	0.2956	0.3316	0.3265
trafficQA-TFNSW	Q1	0.52	0.515	0.5325	0.4125	0.395	0.41	0.5975	0.6475	0.73	0.95	0.965	0.9725
	Q2	0.1633	0.1837	0.1701	0.1293	0.1293	0.1565	0.2721	0.2857	0.3061	0.4354	0.381	0.4762
	Q3	0.2381	0.1973	0.2313	0.2109	0.2041	0.2109	0.4014	0.381	0.4285	0.6871	0.66	0.7279

Table 9: Comparison of Different RAG Methods on WeatherQA-Irish Dataset

WeatherQA-Irish											
Model	Data Type	Question 1		Question 2				Question 3			
		Acc	Hall.	Acc	F1 Score	HR	Hall.	Acc	F1 Score	HR	Hall.
Llama-3-8b	Graph-RAG	0.916	0.086	0.063	0.673	0.509	0.5	0.15	0.684	0.517	0.48
	KG-RAG	0.892	0.108	0.217	0.815	0.694	0.34	0.386	0.809	0.686	0.22
	KG-IRAG	0.927	0.081	0.242	0.821	0.702	0.32	0.415	0.831	0.715	0.22
GPT-3.5-turbo	Graph-RAG	0.958	0.042	0.087	0.673	0.509	0.48	0.14	0.684	0.517	0.42
	KG-RAG	0.945	0.055	0.208	0.807	0.683	0.32	0.362	0.803	0.677	0.18
	KG-IRAG	0.958	0.036	0.222	0.817	0.697	0.32	0.386	0.824	0.703	0.2
GPT-4o-mini	Graph-RAG	0.97	0.03	0.145	0.673	0.509	0.32	0.251	0.684	0.517	0.28
	KG-RAG	0.975	0.025	0.304	0.832	0.718	0.24	0.546	0.813	0.695	0.18
	KG-IRAG	0.983	0.028	0.323	0.84	0.721	0.22	0.585	0.859	0.754	0.16
GPT-4o	Graph-RAG	0.983	0.017	0.208	0.673	0.509	0.24	0.357	0.684	0.517	0.22
	KG-RAG	0.998	0.002	0.435	0.857	0.751	0.14	0.575	0.839	0.728	0.16
	ToG	0.996	0.004	0.449	0.842	0.770	0.15	0.584	0.857	0.736	0.15
	KG-IRAG	0.998	0.002	0.459	0.875	0.783	0.18	0.604	0.88	0.789	0.14

Hit Rate is not considered, since it contain less temporal reasoning compared with Q2 and Q3. The results of WeatherQA-Irish are

shown in table 9, while WeatherQA-Sydney results are shown in table 10, and those of TrafficQA-TFNSW are in table 11.

Table 10: Comparison of Different RAG methods on WeatherQA-Sydney Dataset

WeatherQA-Sydney											
Model	Data Type	Question 1		Question 2				Question 3			
		Acc	Hall.	Acc	F1 Score	HR	Hall.	Acc	F1 Score	HR	Hall.
Llama-3-8b	Graph-RAG	0.877	0.123	0.141	0.514	0.349	0.44	0.167	0.503	0.326	0.42
	KG-RAG	0.908	0.092	0.183	0.762	0.63	0.26	0.226	0.741	0.606	0.24
	KG-IRAG	0.916	0.084	0.192	0.841	0.721	0.24	0.239	0.833	0.711	0.26
GPT-3.5-turbo	Graph-RAG	0.892	0.108	0.139	0.514	0.349	0.46	0.157	0.503	0.326	0.42
	KG-RAG	0.933	0.067	0.205	0.751	0.617	0.24	0.231	0.738	0.602	0.22
	KG-IRAG	0.938	0.062	0.246	0.837	0.716	0.22	0.257	0.844	0.725	0.22
GPT-4o-mini	Graph-RAG	0.947	0.053	0.195	0.514	0.349	0.3	0.188	0.503	0.326	0.36
	KG-RAG	0.973	0.027	0.29	0.775	0.644	0.18	0.337	0.78	0.651	0.2
	KG-IRAG	0.967	0.033	0.352	0.862	0.761	0.16	0.362	0.856	0.757	0.2
GPT-4o	Graph-RAG	0.968	0.032	0.226	0.514	0.349	0.26	0.25	0.503	0.326	0.22
	KG-RAG	0.978	0.022	0.378	0.785	0.657	0.16	0.362	0.793	0.663	0.14
	ToG	0.98	0.009	0.411	0.833	0.726	0.15	0.398	0.824	0.743	0.12
	KG-IRAG	0.983	0.017	0.429	0.852	0.75	0.12	0.411	0.869	0.769	0.12

Table 11: Comparison of Different RAG methods on TrafficQA-TFNSW dataset

TrafficQA-TFNSW											
Model	Data Type	Question 1		Question 2				Question 3			
		Acc	Hall.	Acc	F1 Score	HR	Hall.	Acc	F1 Score	HR	Hall.
Llama-3-8b	Graph-RAG	0.465	0.535	0.095	0.694	0.537	0.48	0.122	0.709	0.548	0.4
	KG-RAG	0.503	0.497	0.218	0.843	0.725	0.2	0.306	0.835	0.716	0.22
	KG-IRAG	0.52	0.48	0.229	0.863	0.759	0.21	0.333	0.851	0.745	0.26
GPT-3.5-turbo	Graph-RAG	0.418	0.582	0.075	0.694	0.537	0.5	0.088	0.709	0.548	0.44
	KG-RAG	0.43	0.57	0.224	0.841	0.722	0.22	0.252	0.833	0.712	0.22
	KG-IRAG	0.46	0.54	0.239	0.858	0.761	0.22	0.279	0.868	0.766	0.26
GPT-4o-mini	Graph-RAG	0.635	0.365	0.238	0.694	0.537	0.36	0.32	0.709	0.548	0.32
	KG-RAG	0.722	0.263	0.374	0.852	0.736	0.14	0.503	0.857	0.743	0.16
	KG-IRAG	0.748	0.251	0.381	0.882	0.785	0.22	0.51	0.894	0.813	0.18
GPT-4o	Graph-RAG	0.93	0.07	0.361	0.694	0.537	0.25	0.565	0.709	0.548	0.26
	KG-RAG	0.963	0.047	0.537	0.878	0.783	0.14	0.776	0.869	0.771	0.12
	ToG	0.967	0.04	0.531	0.861	0.788	0.18	0.789	0.894	0.831	0.17
	KG-IRAG	0.965	0.035	0.558	0.886	0.802	0.2	0.803	0.914	0.843	0.16